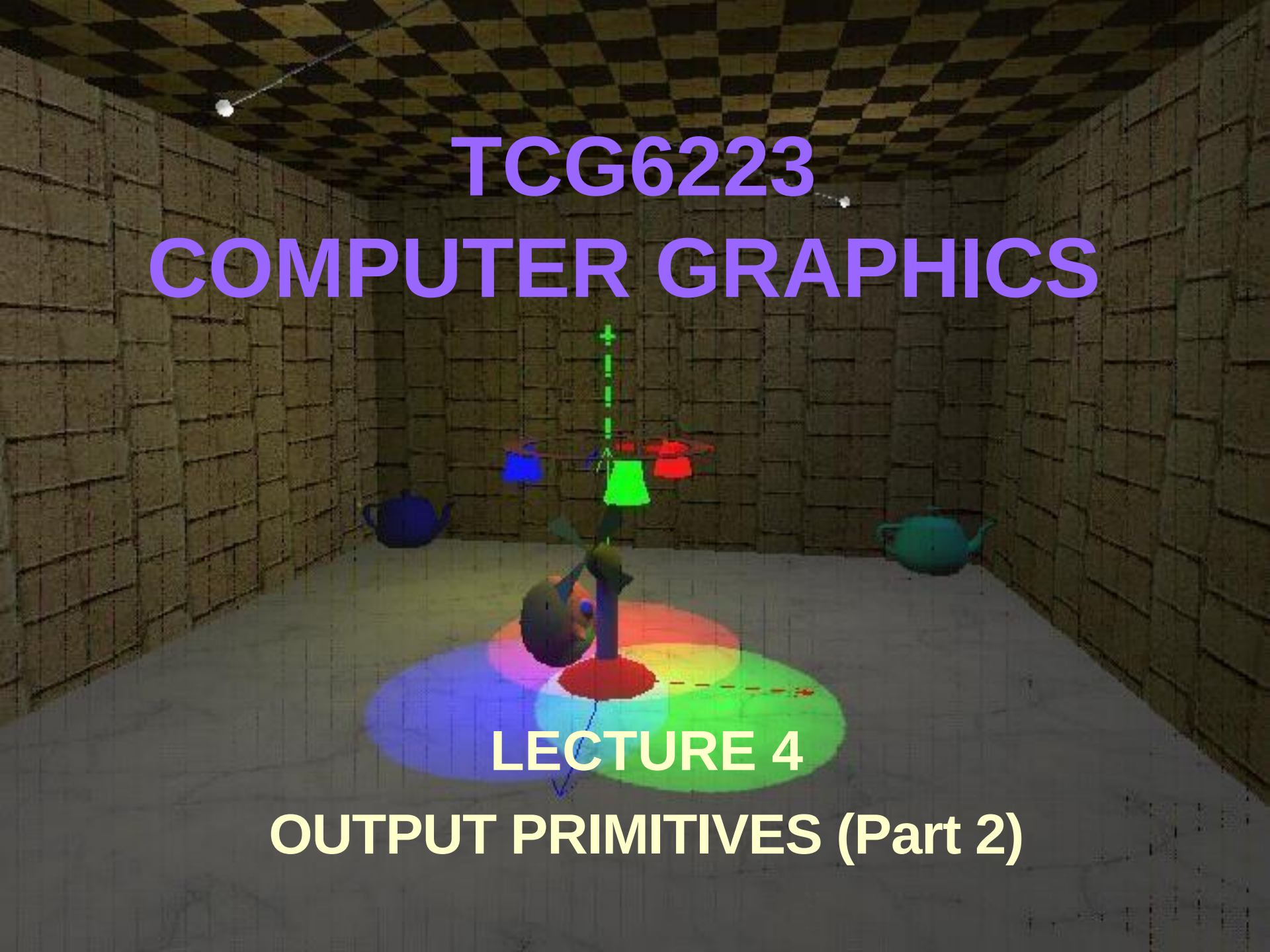




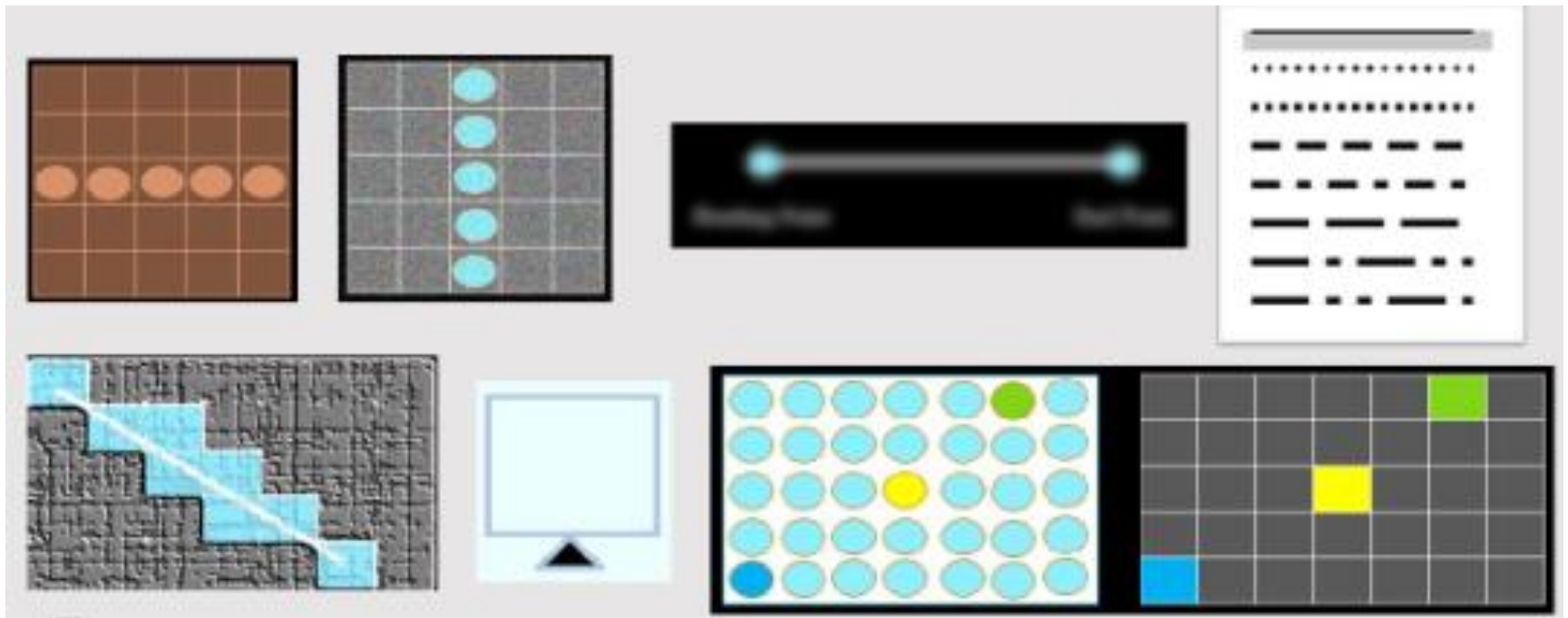
TCG6223

COMPUTER GRAPHICS

LECTURE 4

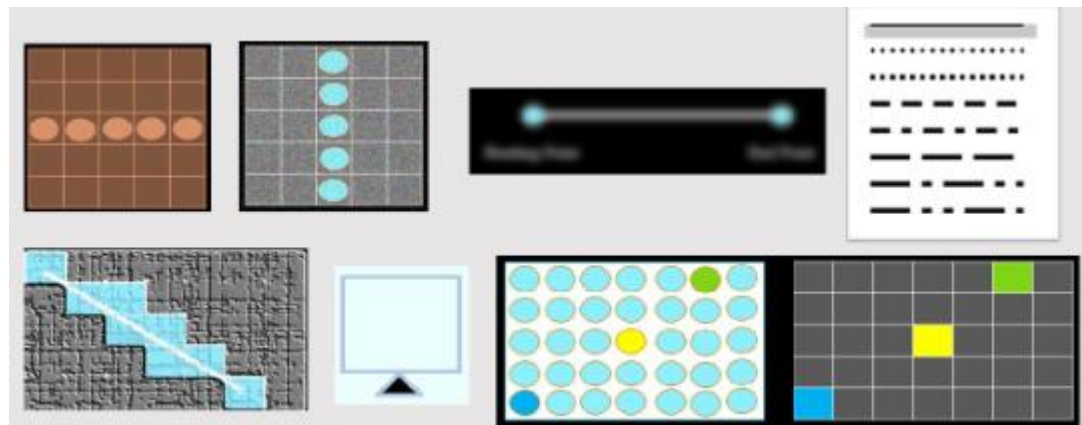
OUTPUT PRIMITIVES (Part 2)

Output Primitives



Output Primitives

- The **basic objects** out of which a graphics display is created are called.
- Describes the geometry of objects and – typically referred to as **geometric primitives**.
- **Examples:** point, line, text, filled region, images, quadric surfaces, spline curves
- Each of the output primitives has its own set of **attributes**.



Content: 2D Graphics Algorithms

- Output Primitives
- Line Drawing Algorithms
 - Analytical Method
 - DDA Algorithm
 - Midpoint Algorithm
 - Bresenham's Algorithm
- Anti-aliasing
- Fill-Area Algorithms

Line Drawing Algorithms

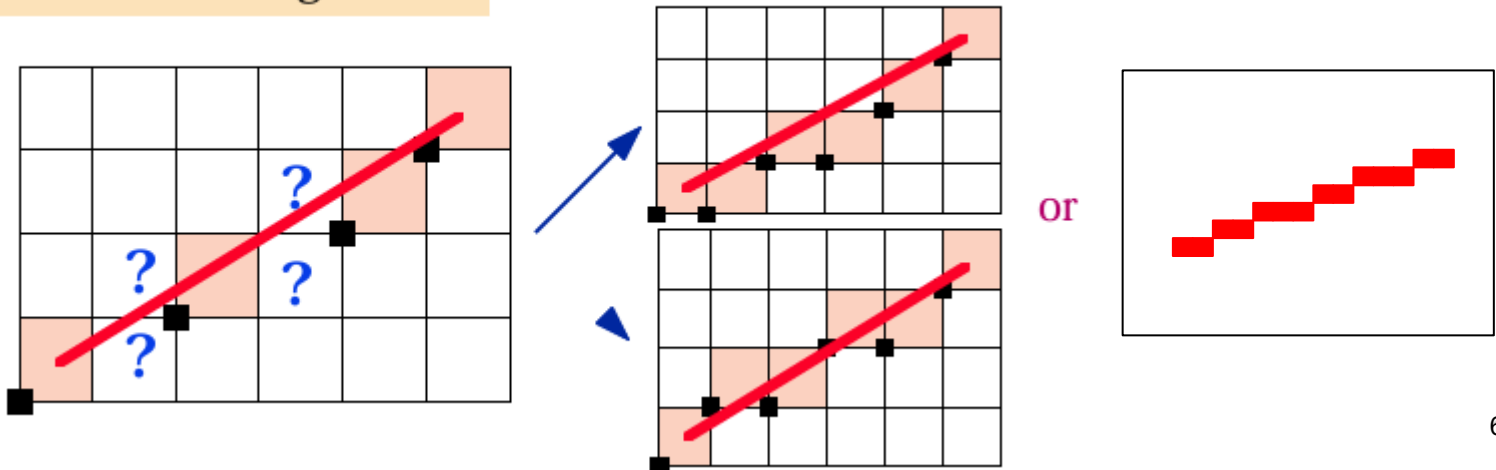
Line Drawing

- Line drawing is **fundamental** to computer graphics.
- We must have **fast and efficient** line drawing functions.

Rasterization Problem: Given only the two end points, how to compute the **intermediate pixels**, so that the set of pixels **closely approximate** the ideal line.

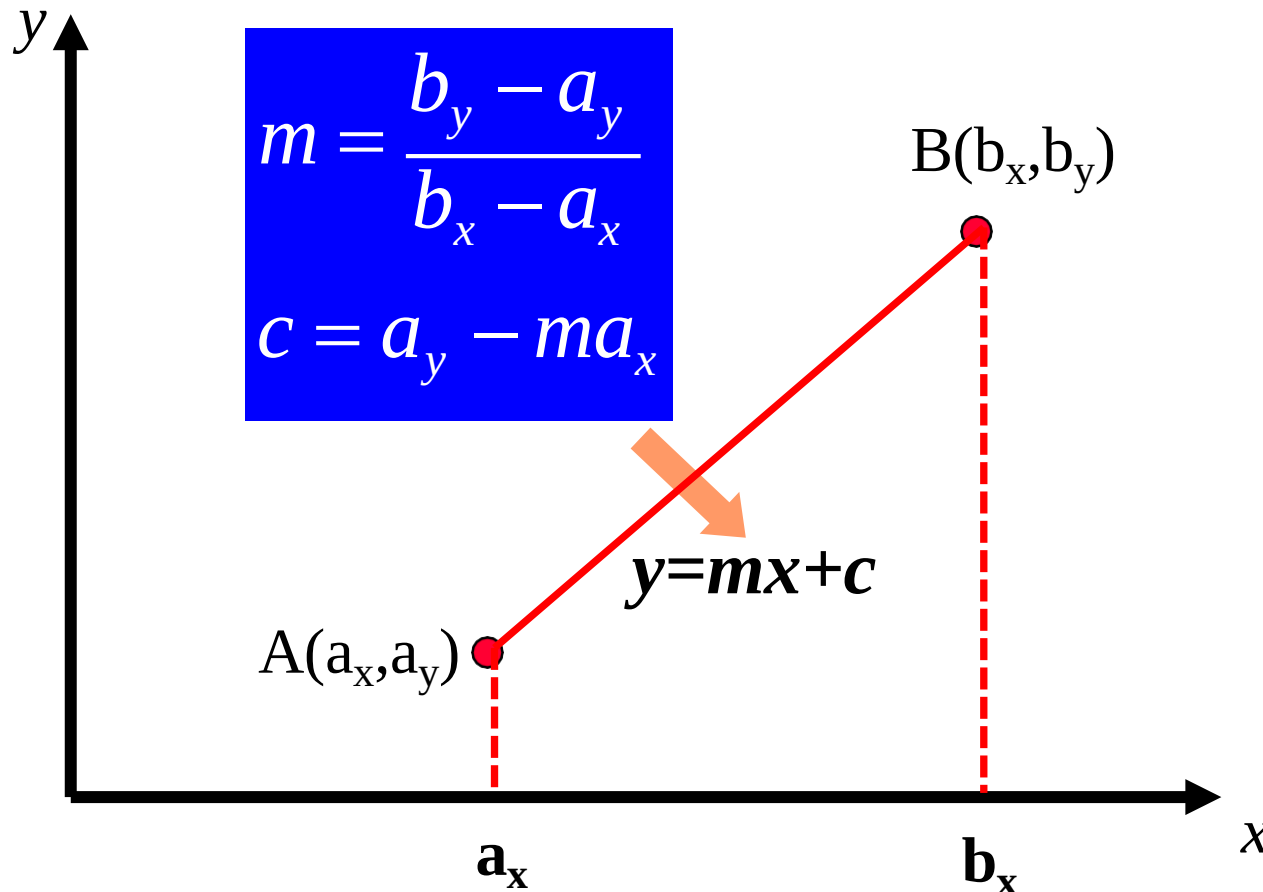
Aliasing occurs when a line appears jagged or stair-stepped instead of smooth = visual distortion that occur when representing a continuous signal (such as a smooth line or curve) in a discrete format.

Rasterization of straight lines.



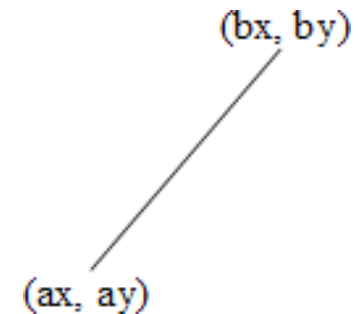
Line Drawing - Analytical Method

Straightforward approach that utilize **mathematical equations** to represent and render lines on a raster grid. However, it may be **computationally more intensive** compared to incremental methods like Bresenham's algorithm.



Line Drawing - Analytical Method

```
//Given (ax, ay) and (bx, by)  
  
double m = (double) (by-ay) / (bx-ax) ;  
double c = ay - m*ax;  
double y;  
int iy;  
for (int x=ax; x<=bx; x++) {  
    y = m*x + c;  
    iy = round(y) ;  
    setPixel(x, iy) ;  
  
}
```



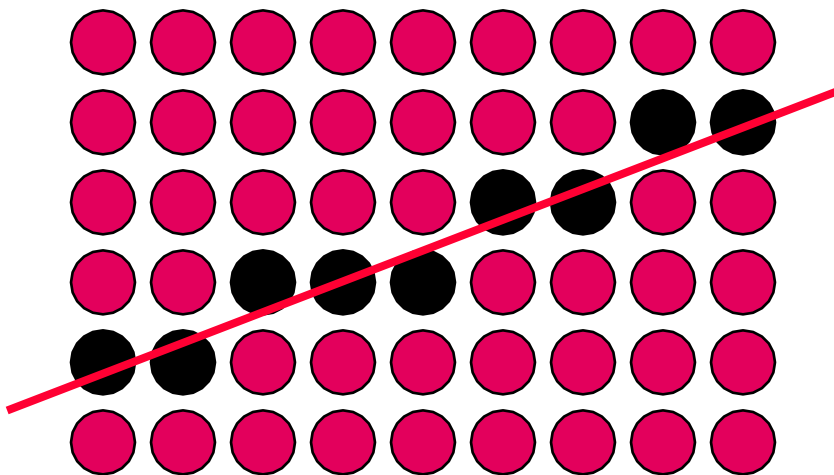
- Directly based on the **analytical equation** of a line.
- Involves **floating point multiplication** and **addition**
- Requires **round-off function**.

Incremental Algorithms

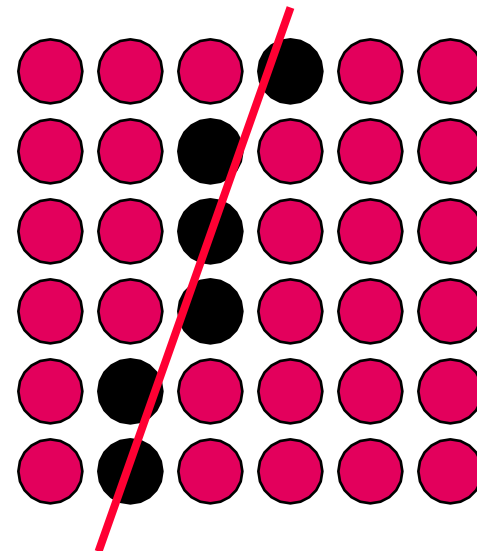
**I have got a pixel on the line (Current Pixel).
How do I get the next pixel on the line?**

Compute one point based on the previous point:

$(x_0, y_0) \dots\dots\dots (x_k, y_k) \xrightarrow{\text{purple arrow}} (x_{k+1}, y_{k+1}) \dots\dots\dots$

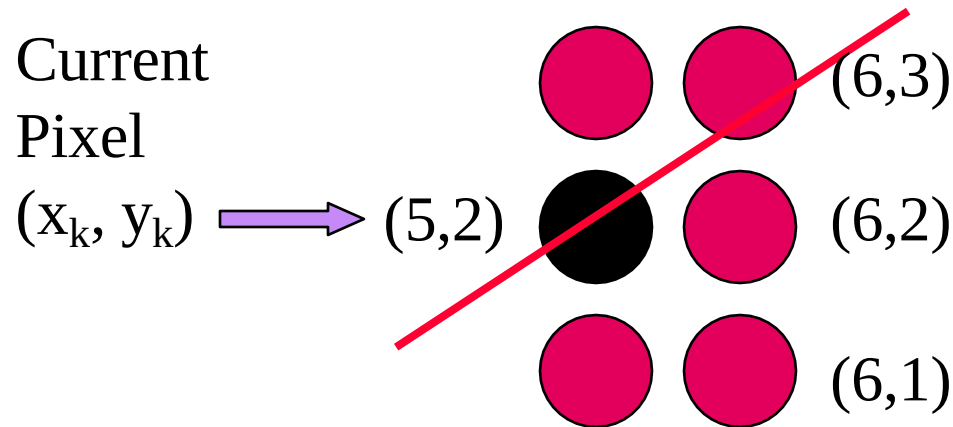


**Next pixel on next column
(when slope is small)**



**Next pixel on next row
(when slope is large)**

Incrementing along x



To find (x_{k+1}, y_{k+1}) :

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = ?$$

- Assumes that the next pixel to be set is on the next column of pixels (Incrementing the value of x !)
- Not valid if slope of the line is large

Line Drawing - DDA

Digital Differential Analyzer Algorithm is an **incremental algorithm** employs floating-point arithmetic.

Assumption: Slope (m) is **less** than 1 (**Increment along x**).

Current Pixel = (x_k, y_k) .

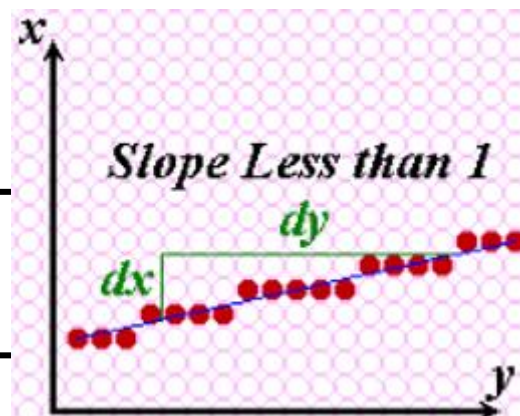
(x_k, y_k) lies on the given line. $\longrightarrow y_k = m.x_k + c$

Next pixel is on **next column**. $\longrightarrow x_{k+1} = x_k + 1$

Next point (x_{k+1}, y_{k+1}) on the line $\longrightarrow y_{k+1} = m.x_{k+1} + c$
 $= m(x_k + 1) + c$
 $= y_k + m$

Given a point (x_k, y_k) on a line,
next point is given by

$$x_{k+1} = x_k + 1$$
$$y_{k+1} = y_k + m$$



Line Drawing - DDA

Assumption: Slope is **greater** than 1 (**Increment along y**).
Current Pixel = (x_k, y_k) .

(x_k, y_k) lies on the given line.

Next pixel is on next **row**.

Next point (x_{k+1}, y_{k+1}) on the line

$$\longrightarrow y_k = m \cdot x_k + c$$

$$\longrightarrow y_{k+1} = y_k + 1$$

$$\longrightarrow y_k = m x_k + c$$

$$x_k = \frac{y_k}{m} + \frac{c}{m} \dots\dots(1)$$

$$x_{k+1} = \frac{y_{k+1}}{m} + \frac{c}{m} \dots\dots(2)$$

$$(2) - (1) \Rightarrow x_{k+1} - x_k = \frac{y_{k+1} - y_k}{m}$$

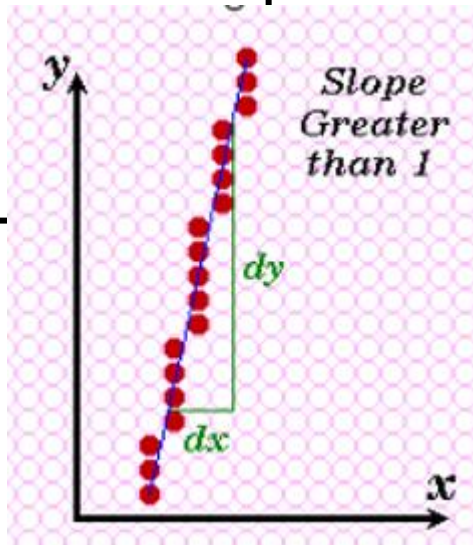
$$x_{k+1} = x_k + \frac{1}{m}$$

inverse slope 20

Given a point (x_k, y_k) on a line,
next point is given by

$$x_{k+1} = x_k + 1/m$$

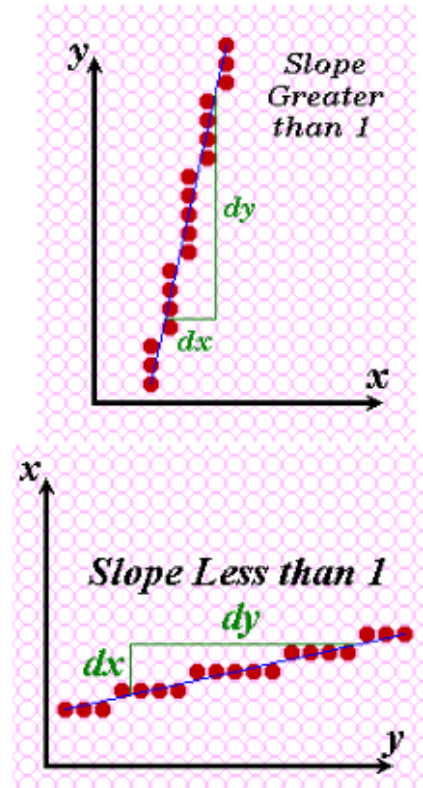
$$y_{k+1} = y_k + 1$$



Line Drawing – DDA ($m < 1$)

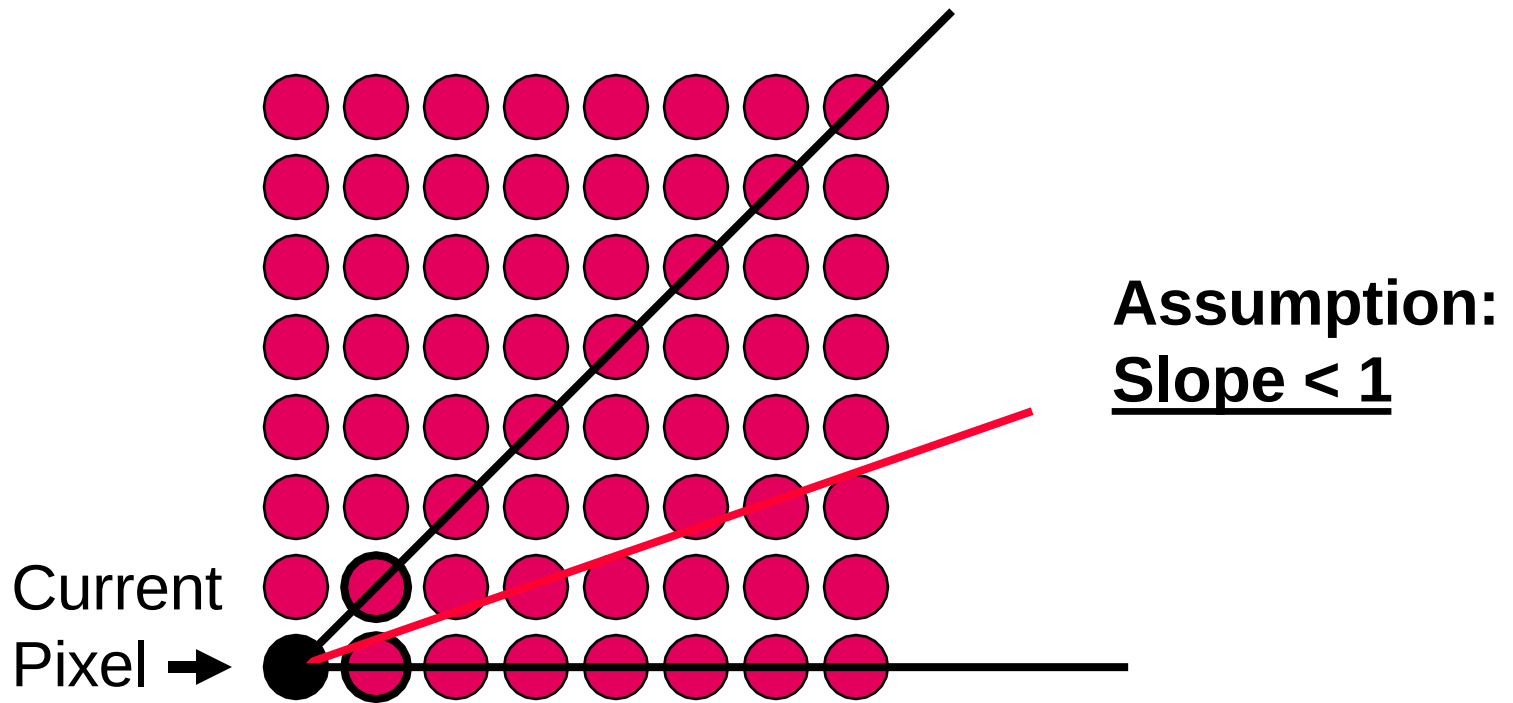
```
double m = (double) (by-ay) / (bx-ax) ;  
double y = ay;  
  
    int iy;  
    for (int x=ax; x<=bx; x++) {  
        iy = round(y) ;  
        setPixel(x, iy) ;  
        y += m;  
    }
```

- Does **not** involve any floating point multiplication
- Involves **floating point addition**.
- Advantage: simplicity and ease of implementation
- Disadvantage: less efficient than Bresenham's algorithm because of the need for floating-point calculations, which can be computationally expensive compared to integer arithmetic



Midpoint Algorithm

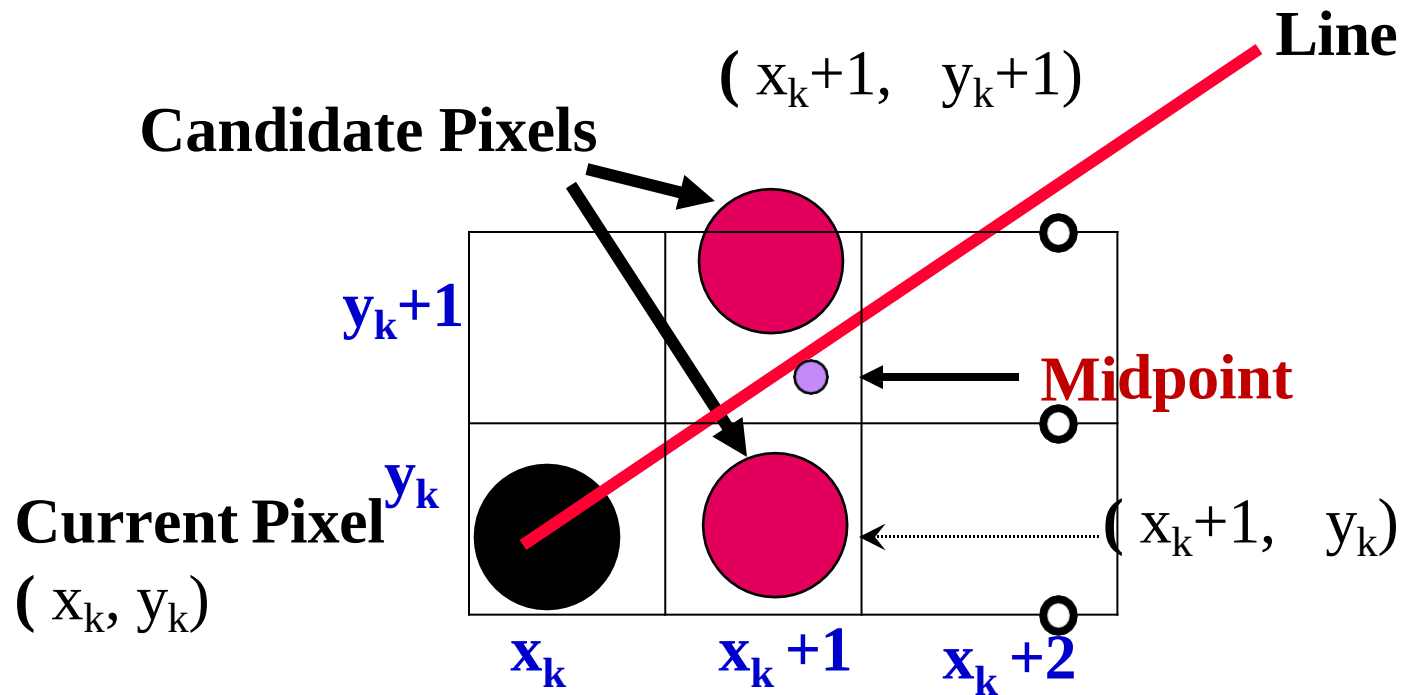
Midpoint algorithm is an incremental algorithm



$$x_{k+1} = x_k + 1$$

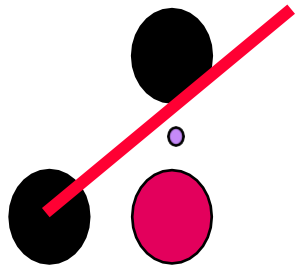
$$y_{k+1} = \text{Either } y_k \text{ or } y_k + 1$$

Midpoint Algorithm - Notations

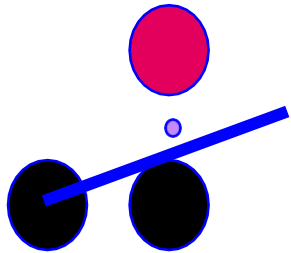


Coordinates of Midpoint = $(x_k+1, y_k+(1/2))$

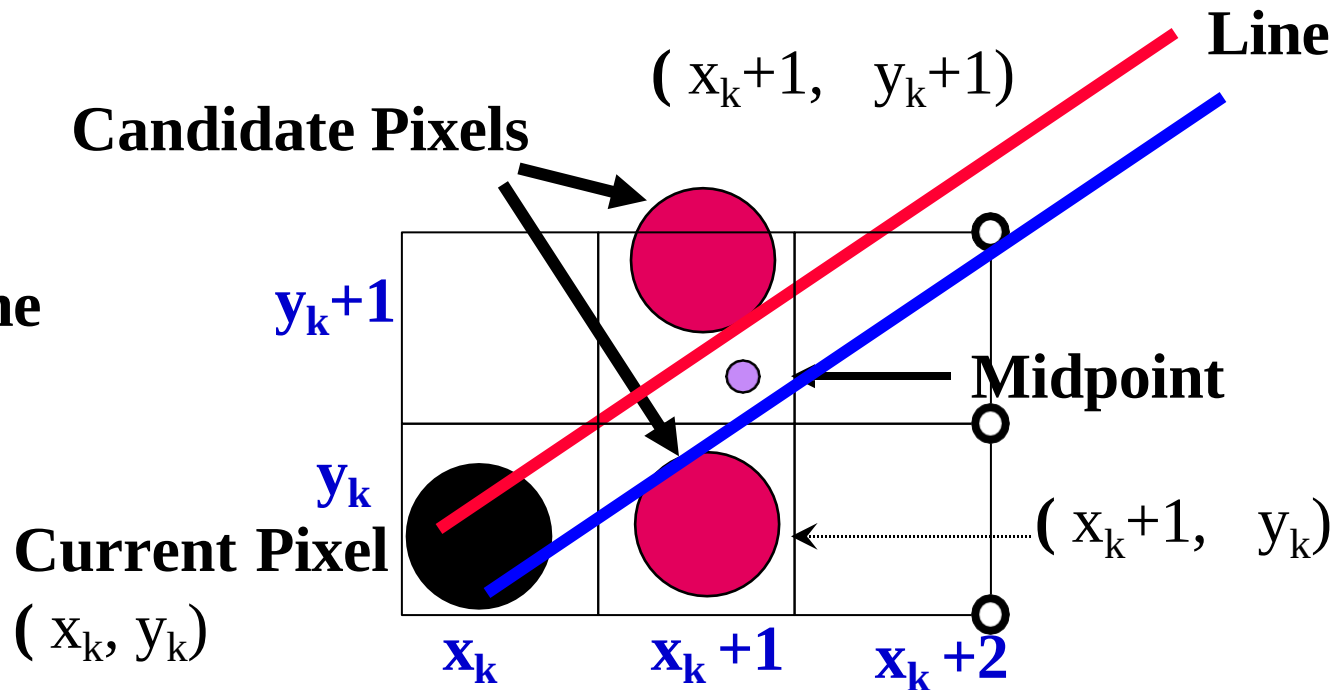
Midpoint Algorithm: Choice of the next pixel



Midpoint Below Line



Midpoint Above Line



If the midpoint is below the line, then the next pixel is (x_k+1, y_k+1) .

If the midpoint is above the line, then the next pixel is (x_k+1, y_k) .

Equation of a line revisited

Equation of the line:

$$\frac{y - a_y}{b_y - a_y} = \frac{x - a_x}{b_x - a_x}$$

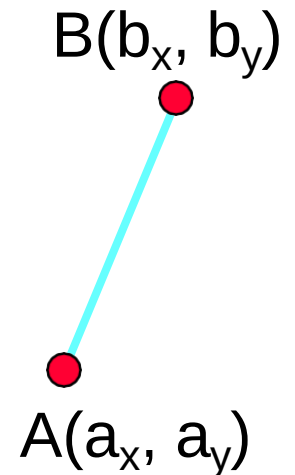
Let $w = b_x - a_x$, and $h = b_y - a_y$

Then, $h(x - a_x) - w(y - a_y) = 0$

(h, w, a_x, a_y are all integers)

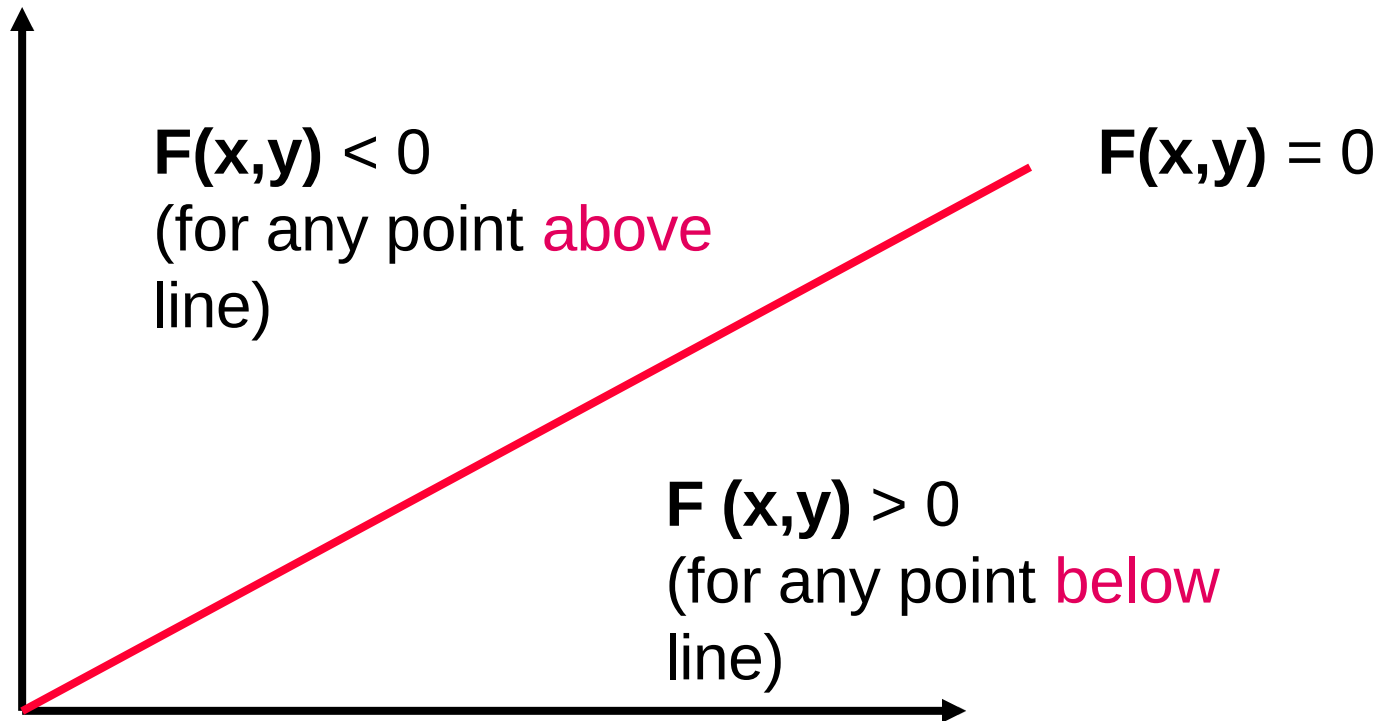
In other words, every point (x, y) on the line satisfies the equation $F(x, y) = 0$, where

$$F(x, y) = h(x - a_x) - w(y - a_y)$$

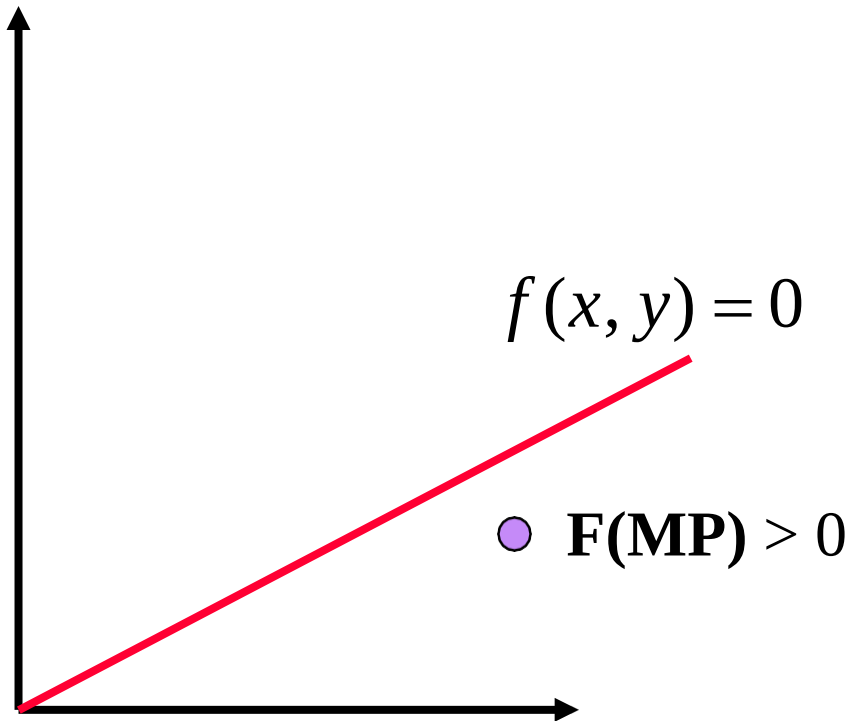


Midpoint Algorithm:

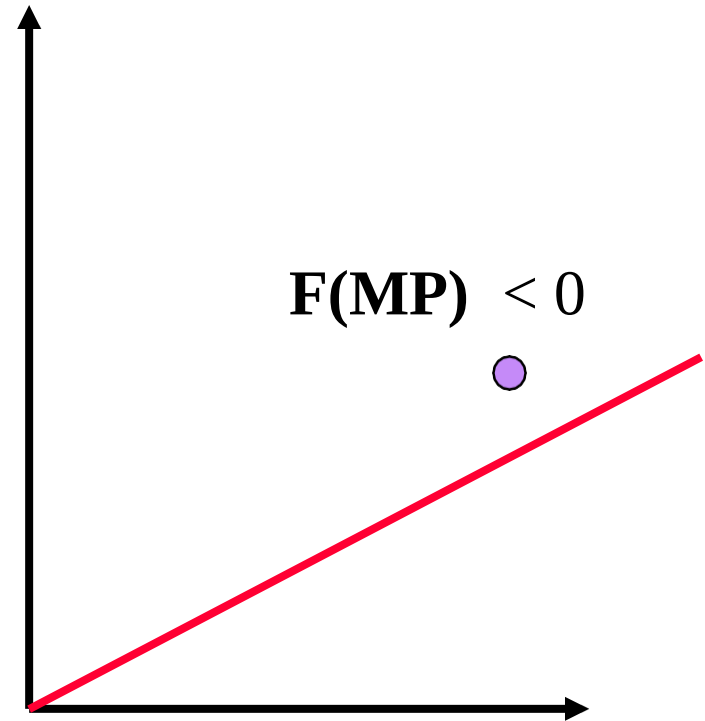
Regions below and above the line



Midpoint Algorithm: Decision Criteria



Midpoint below line

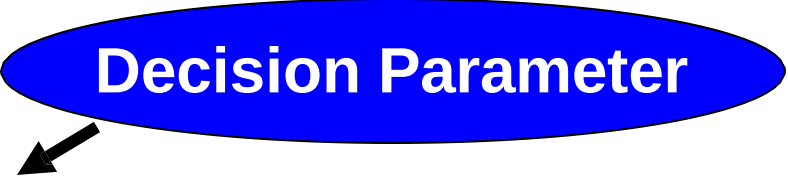


Midpoint above line

Midpoint Algorithm

Decision Criteria

Decision Parameter

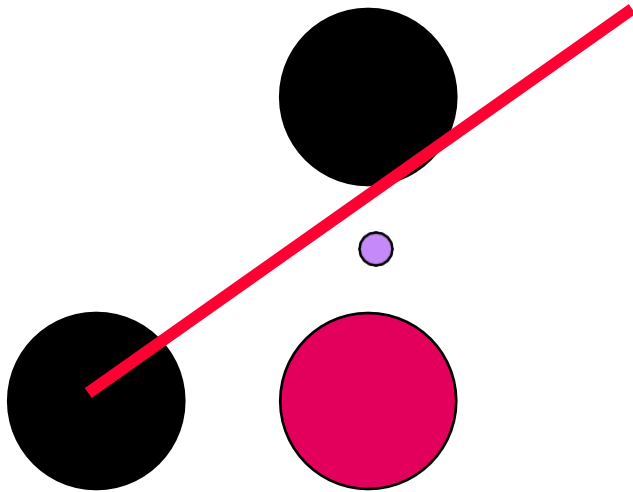


$$F(MP) = F(x_k+1, y_k + \frac{1}{2}) = F_k \quad (Notation)$$

If $F_k < 0$: The midpoint is above the line. So the next pixel is (x_k+1, y_k)

If $F_k \geq 0$: The midpoint is below or on the line. So the next pixel is (x_k+1, y_k+1)

Midpoint Algorithm – Story so far

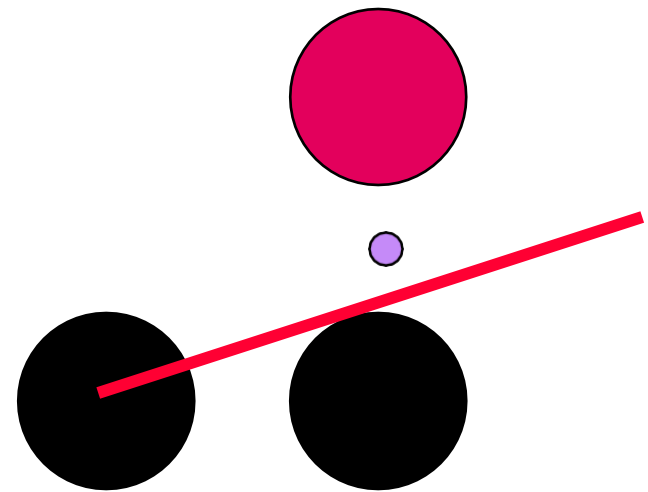


Midpoint Below Line

$$F_k \geq 0$$

$$y_{k+1} = y_k + 1$$

Next pixel = $(x_k + 1, y_k + 1)$



Midpoint Above Line

$$F_k < 0$$

$$y_{k+1} = y_k$$

Next pixel = $(x_k + 1, y_k)$

Midpoint Algorithm: Update Equation

$$F_k = F(x_k+1, y_k + 1/2) = h (x_k+1 - a_x) - w (y_k + 1/2 - a_y) \quad \longrightarrow \textcircled{1}$$

$$F_{k+1} = F(x_{k+1}+1, y_{k+1} + 1/2) = h (x_{k+1}+1 - a_x) - w (y_{k+1} + 1/2 - a_y) \quad \longrightarrow \textcircled{2}$$

$$\begin{aligned} \textcircled{2} - \textcircled{1} &\longrightarrow F_{k+1} - F_k = h (x_{k+1} - x_k) - w (y_{k+1} - y_k) \\ F_{k+1} - F_k &= h (x_k + 1 - x_k) - w (y_{k+1} - y_k) \\ F_{k+1} &= F_k + h - w (y_{k+1} - y_k) \end{aligned}$$

Thus, given $F_k < 0$, $y_{k+1} = y_k$ then $F_{k+1} = F_k + h$

given $F_k \geq 0$, $y_{k+1} = y_k + 1$ then $F_{k+1} = F_k + h - w$

$$F_0 = h (a_x + 1 - a_x) - w (a_y + 1/2 - a_y) = h - w/2$$

Midpoint Algorithm: Update Equation

Summary:

Update Equation



$$F_{k+1} = F_k + h - w (y_{k+1} - y_k)$$

given $F_k < 0$, $y_{k+1} = y_k$ then $F_{k+1} = F_k + h$

given $F_k \geq 0$, $y_{k+1} = y_k + 1$ then $F_{k+1} = F_k + h - w$

$$F_0 = h - w/2$$

Midpoint Algorithm

```
int    h = by-ay;
int    w = bx-ax;
float  F = h-w/2;
int    y = ay;
for (int x=ax; x<=bx; x++) {
    setPixel(x, y);

    if(F < 0)
        F += h;
    else {
        F += h-w;
        y++;
    }
}
```


Bresenham's Algorithm

(Improved Midpoint Algorithm)

$$\begin{aligned} F_k < 0 & : y_{k+1} = y_k \\ F_k \geq 0 & : y_{k+1} = y_k + 1 \end{aligned}$$

$$\begin{aligned} \text{Hence, } F_{k+1} &= F_k + 2h \\ \text{Hence, } F_{k+1} &= F_k + 2(h - w) \end{aligned}$$

$$F_0 = 2h - w$$

In both cases, only the **integer** decision parameter F is evaluated, avoiding the need for floating-point arithmetic.

Bresenham's Algorithm

(Improved Midpoint Algorithm)

```
int    h  = by-ay;
int    w  = bx-ax;
int    F  = 2*h-w;
int    y  = ay;
for (int x=ax; x<=bx; x++) {
    setPixel(x, y);

    if(F < 0)
        F += 2*h;
    else {
        F += 2*(h-w);
        y++;
    }
}
```

Anti-aliasing

Anti-aliasing

Anti-aliasing is not a line drawing algorithm in itself, used to diminish the jagged edges of an image or a line, so that the line appears to be smoother; by changing the pixels around the edges to intermediate colors or gray scales.

Method: **Supersampling, Multi-Sampling, Filtering, Alpha Blending**

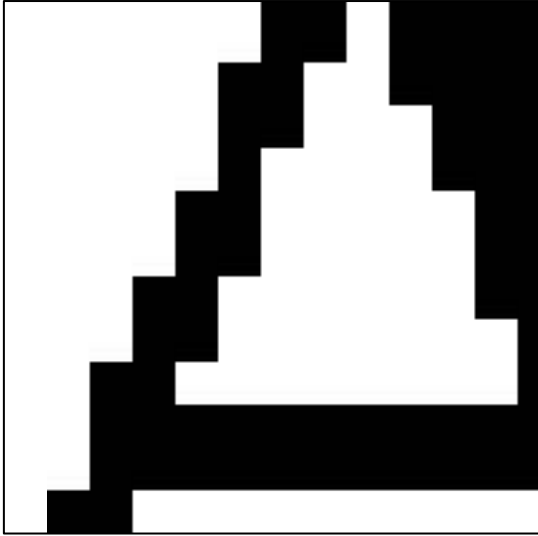
E.g. Anti-aliasing disabled:

Antialiasing

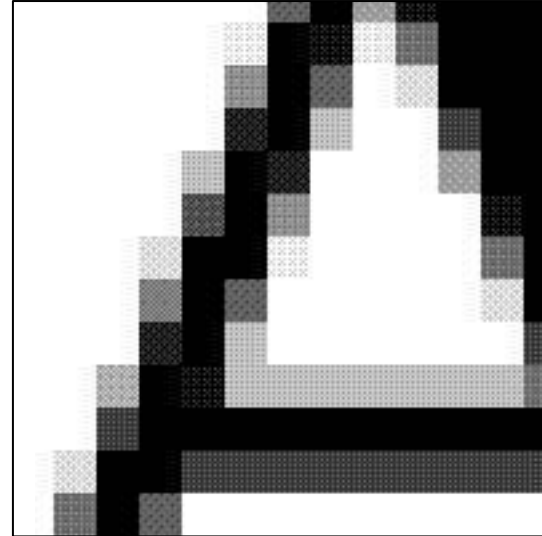
E.g. Anti-aliasing enabled:

Antialiasing

Anti-aliasing (OpenGL)



Anti-aliasing disabled



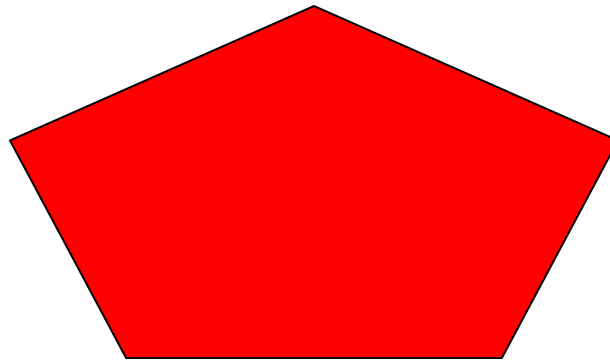
Anti-aliasing enabled

Setting anti-aliasing option for lines:
`glEnable (GL_LINE_SMOOTH);`

Fill Area Algorithms

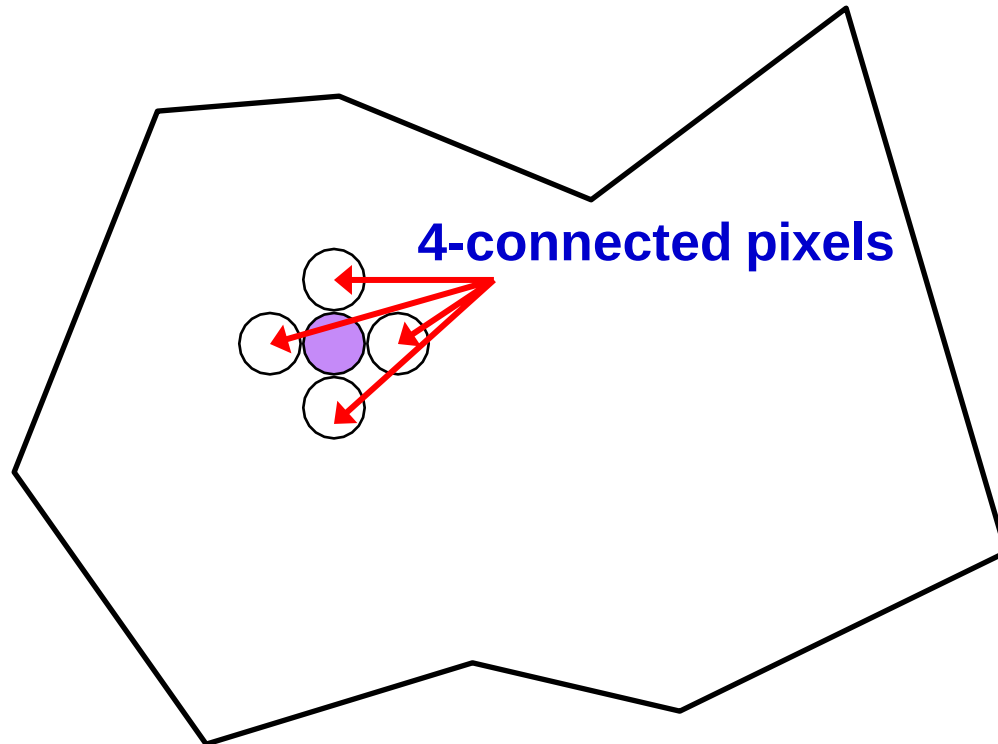
Fill Area Algorithms

- Fill-Area algorithms are used to fill the interior of a polygonal shape.
- Many algorithms perform fill operations by first identifying the interior points, given the polygon boundary.



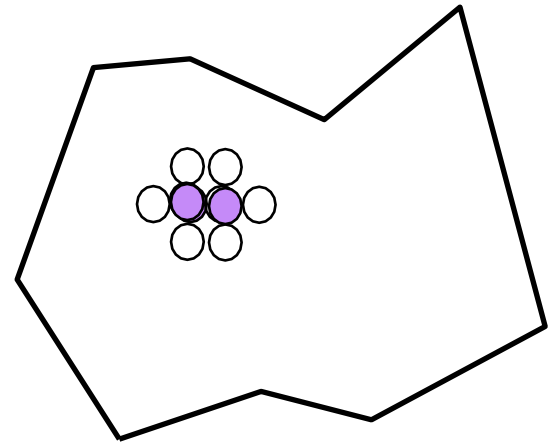
Basic Filling Algorithm

The basic filling algorithm is commonly used in interactive graphics packages, where the user specifies an interior point of the region to be filled.



Basic Filling Algorithm

- 1 Set the user specified point.
- 2 Store the **four neighboring pixels** in a stack.
- 3 Remove a pixel from the **stack**.
- 4 If the pixel is not set,
 - Set the pixel
 - Push its four neighboring pixels into the stack
- 5 Go to step 3
- 6 Repeat till the stack is empty.

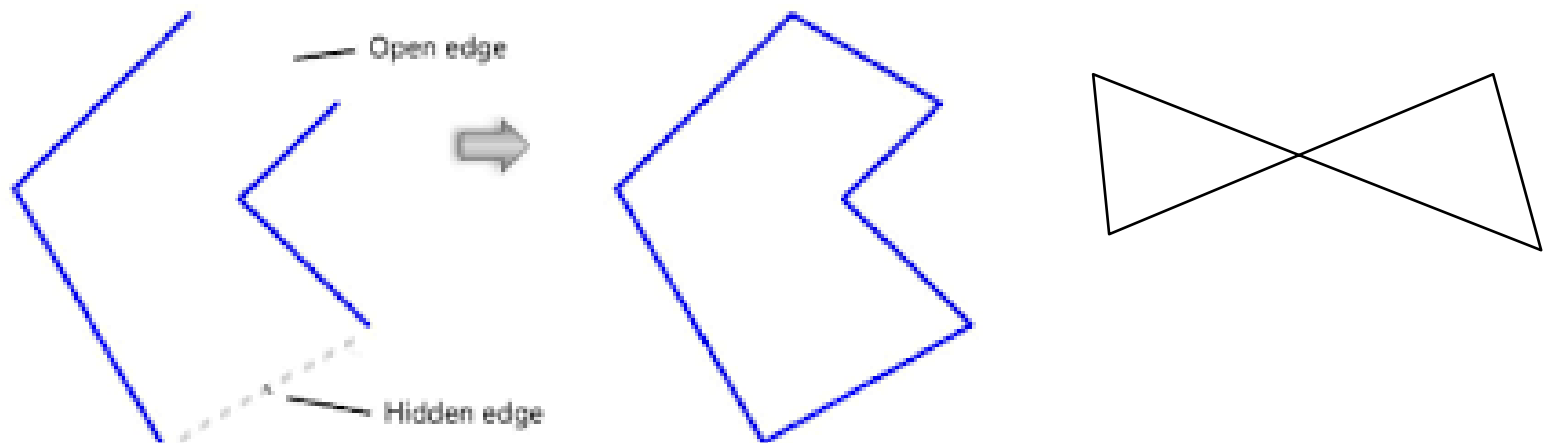


Basic Filling Algorithm (Code)

```
void fill(int x, int y) {  
    if(getPixel(x,y)==0){  
        setPixel(x,y);  
        fill(x+1,y);  
        fill(x-1,y);  
        fill(x,y+1);  
        fill(x,y-1);  
    }  
}
```

Basic Filling Algorithm: Conditions

- Requires an **interior point**.
- Involves considerable **amount of stack** operations.
- The boundary has to be **closed**.
- Not suitable for **self-intersecting** polygons



Types of Basic Filling Algorithms

■ Boundary Fill Algorithm (Seed Fill)

- For filling a region with a single boundary color.
- Choose seed **within the region** to be filled
- Condition for setting pixels:
 - Color is **not the same** as border color
 - Color is **not the same** as fill color

■ Flood Fill Algorithm (4-connected "or" 8-connected" fill)

- For filling a region with multiple boundary colors.
- Fills pixels based on a target color (often the background color) and a fill color
- Condition for setting pixels:
 - Color is **same** as the old interior color

Boundary Fill Algorithm (Code)

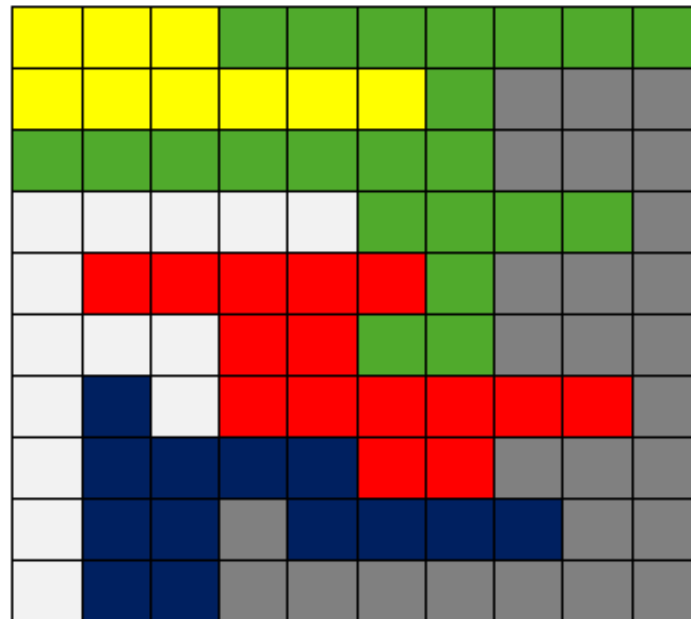
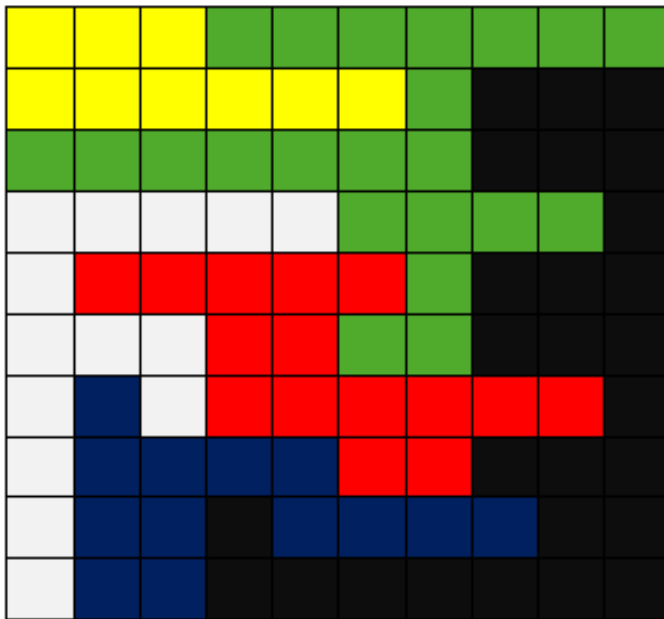
```
void boundaryFill(int x, int y,  
                  int fillColor, int borderColor)  
{  
    /* Set current color to fillColor, then perform the following operations. */  
    getPixel(x, y, color);  
    if ((color != borderColor)  
        && (color != fillColor)) {  
        setPixel(x,y); // Set color of pixel to fillColor.  
        boundaryFill(x+1,y,fillColor,borderColor);  
        boundaryFill(x-1,y,fillColor,borderColor);  
        boundaryFill(x,y+1,fillColor,borderColor);  
        boundaryFill(x,y-1,fillColor,borderColor);  
    }  
}
```

Flood Fill Algorithm (Code)

```
void floodFill(int x, int y,  
               int fillColor, int oldColor)  
{  
    /* Set current color to fillColor, then perform the following operations. */  
    getPixel(x, y, color);  
    if (color == oldColor)  
    {  
        setPixel(x,y) ; // Set color of pixel to fillColor.  
        floodFill(x+1, y, fillColor, oldColor);  
        floodFill(x-1, y, fillColor, oldColor);  
        floodFill(x, y+1, fillColor, oldColor);  
        floodFill(x, y-1, fillColor, oldColor);  
    }  
}
```

Flood Fill Algorithm

Consider the following matrix to the left – if the start node is (3, 9), target color is “**BLACK**” and replacement color is “**GREY**”, the algorithm looks for all nodes in the matrix that are connected to the start node by a path of the target color and changes them to the replacement color.



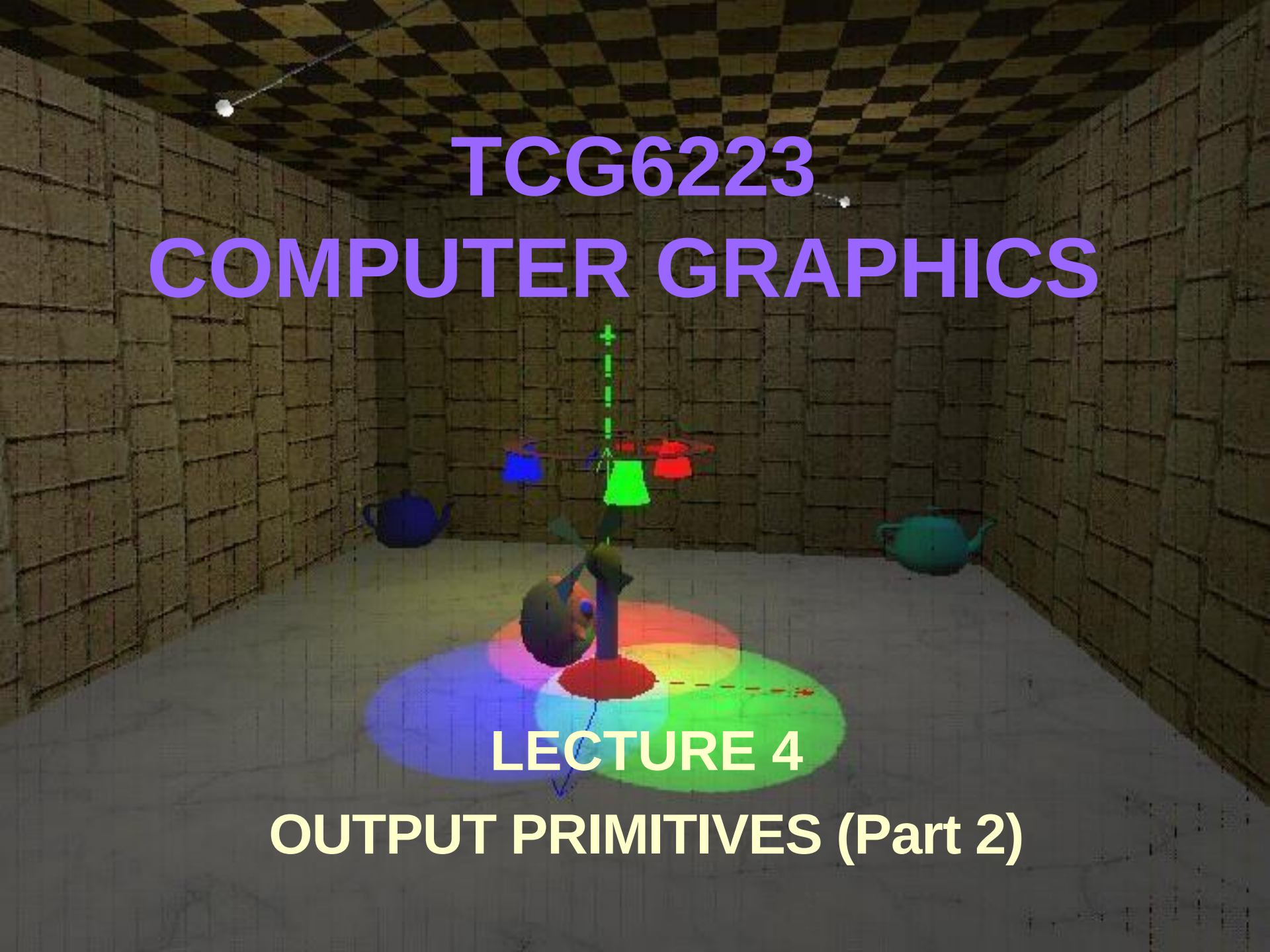
Filling Polygons (OpenGL)

Enabling polygon fill (Default):

```
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

Disabling polygon fill:

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
```

TCG6223

COMPUTER GRAPHICS

LECTURE 4

OUTPUT PRIMITIVES (Part 2)